ARTIFICIAL INTELLIGENCE (CT 653)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 7: Applications of AI

Syllabus Hours: 14 hrs | Marks Weight: 26 marks (Largest Chapter!)

Complete Exam-Ready Answers | All Past Questions (2070–2082)

CHAPTER 7 SYLLABUS OVERVIEW

Sub-topic	Topics	Typical Marks
7.1 Neural Networks	Network structure, Adaline, Perceptron, Multilayer Perceptron, Back Propagation, Hopfield, Kohonen	10–14
7.2 Expert System	Architecture, Knowledge acquisition, Declarative vs Procedural knowledge, Development stages	6–8
7.3 NLP & Machine Vision	Levels: Phonetic, Syntactic, Semantic, Pragmatic; NLP steps; Machine Vision steps	6–8

Prepared by: Santosh Kushwaha (ACE079BCT)

SECTION 1: Neural Networks — Structure, McCulloch-Pitts Model & Activation Functions

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin Back, 2079 Jestha Back, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Magh, 2070 Magh

Q1. What is a neural network? Compare ANN with the human brain. Explain the McCulloch-Pitts model of a neuron with a diagram. [Most Repeated] **[3+4]** (2082 Kartik, 2081 Chaitra, 2079 Chaitra, 2076 Baisakh, 2075 Baisakh)

Answer: Artificial Neural Network (ANN)

An Artificial Neural Network (ANN) is a computational model inspired by the structure and functioning of the biological brain. It consists of interconnected processing units called ARTIFICIAL NEURONS (nodes) that work together to learn patterns from data.

ANN vs Human Brain — Comparison

Feature	Human Brain	Artificial Neural Network
Basic unit	Biological neuron (nerve cell)	Artificial neuron (mathematical node)
Number of neurons	~86 billion neurons	Hundreds to billions (in modern deep nets)
Connections	~100 trillion synaptic connections	Millions to billions of weighted edges
Processing speed	~200 Hz (slow)	Billions of operations per second
Learning mechanism	Synaptic plasticity (Hebbian learning)	Backpropagation, gradient descent
Fault tolerance	Very high — redundant connections	Moderate — depends on architecture
Energy consumption	~20 watts (extremely efficient)	Kilowatts to megawatts (large models)
Parallelism	Massively parallel (all neurons active)	Parallel (GPU acceleration)
Adaptability	Learns from very few examples	Needs large datasets
Memory	Distributed across synaptic weights	Distributed in connection weights

McCulloch-Pitts (MP) Neuron Model (1943)

Proposed by: Warren McCulloch (neuroscientist) and Walter Pitts (mathematician) in 1943 — the first mathematical model of a neuron.

Structure:

- INPUTS: x1, x2, ..., xn — binary values (0 or 1)
- WEIGHTS: w1, w2, ..., wn — each connection has a fixed weight (+1 for excitatory, -1 for inhibitory)
- SUMMATION: Computes weighted sum: NET = Σ(wi × xi)
- THRESHOLD (θ): A predefined threshold value
- OUTPUT: y = 1 if NET ≥ θ, else y = 0 (binary step function)

Diagram (text representation):

```
x1 ──(w1)──\
x2 ──(w2)───>  [Σ NET = Σ wi·xi] ──> [θ] ──> Output y
x3 ──(w3)──/

y = 1  if  NET ≥ θ
y = 0  if  NET < θ
```

Example — AND gate using MP Neuron: Inputs x1, x2 □ {0,1}, weights w1=w2=1, threshold θ=2

x1	x2	NET = x1+x2	θ = 2	Output y = AND(x1,x2)
0	0	0	2	0
0	1	1	2	0
1	0	1	2	0
1	1	2	2	1

Limitations of MP Neuron:

- Weights and threshold are FIXED — the neuron cannot learn from data.
- Inputs must be BINARY (0 or 1) — cannot handle real-valued inputs.
- Cannot represent non-linear decision boundaries.
- No learning algorithm was specified — it was a theoretical model only.

Q2. Explain the role of activation function in ANN. How to choose the best activation function for a particular project? [3+4] (2081 Chaitra, 2079 Chaitra, 2078 Chaitra)

Answer: Activation Functions in ANN

Role of Activation Function: The activation function introduces NON-LINEARITY into the neural network, allowing it to learn complex, non-linear patterns. Without it, a multi-layer network collapses into a single linear transformation, making depth useless.

The activation function determines:

- Whether a neuron fires (activates) or not.
- The output range of each neuron.
- The gradient flow during backpropagation (whether learning signals vanish or explode).

Types of Activation Functions:

Function	Formula	Output Range	Graph Shape	Key Property
Step (Binary)	y=1 if x≥θ, else 0	[0,1]	Step at threshold	Used in MP Neuron; not differentiable
Sigmoid (Logistic)	σ(x) = 1/(1+e^(-x))	(0,1)	S-shaped curve	Smooth; probabilistic output; suffers

				vanishing gradient
Tanh	tanh(x) = (e^x - e^(-x))/(e^x + e^(-x))	(-1,1)	S-shaped, zero-centered	Zero-centered; better than sigmoid but still vanishing gradient
ReLU	f(x) = max(0, x)	[0,∞)	Ramp (0 then linear)	Simple, fast; solves vanishing gradient; dying ReLU problem
Leaky ReLU	f(x) = max(0.01x, x)	(-∞,∞)	Small slope for x<0	Fixes dying ReLU
Softmax	σ(xi) = e^xi / Σe^xj	(0,1), sums to 1	Probability distribution	Used in output layer for multi-class classification
Linear	f(x) = x	(-∞,∞)	Straight line	No squashing; used in regression output layer

How to Choose the Best Activation Function:

Situation / Layer	Recommended Function	Reason
Hidden layers (general)	ReLU	Fast, simple, avoids vanishing gradient, works well in practice
Hidden layers (risk of dying ReLU)	Leaky ReLU or ELU	Allows small gradient for negative inputs
Output layer — Binary classification	Sigmoid	Output between 0–1 interpreted as probability
Output layer — Multi-class classification	Softmax	Outputs sum to 1; represents probability distribution over classes
Output layer — Regression	Linear (no activation)	Unbounded output needed for continuous values
Shallow networks / simple problems	Sigmoid or Tanh	Historically used; acceptable for small networks
Deep networks (10+ layers)	ReLU and variants	Vanishing gradient is critical — must use ReLU
RNNs / LSTMs	Tanh, Sigmoid	Standard for recurrent architectures

SECTION 2: Perceptron, Hebbian Learning & Logic Gates

Exam Ref: 2082, 2081, 2080, 2079, 2078, 2077, 2076, 2075, 2074, 2073, 2072, 2071, 2070

Q3. What is a perceptron? Can a single perceptron implement XOR gate? Justify and propose a neural network model for XOR. [Most Repeated] **[1+8]** (2082 Kartik, 2079 Ashwin Back, 2078 Baishakh, 2077 Chaitra, 2075 Baisakh)

Answer: Perceptron

Perceptron: A Perceptron (Frank Rosenblatt, 1957) is the simplest trainable neural network — a single artificial neuron with ADJUSTABLE weights and a learning algorithm. Unlike the MP neuron, the perceptron can LEARN from data.

Structure:

- Inputs: x0=1 (bias), x1, x2, ..., xn
- Weights: w0 (bias weight), w1, w2, ..., wn
- NET: NET = Σ(wi × xi) = w0 + w1x1 + w2x2 + ... + wnxn
- Output: y = 1 if NET ≥ 0, else y = -1 (or 0)

Perceptron Learning Rule: Adjust weights when prediction is wrong:

```
Δwi = η × (target - output) × xi
wi(new) = wi(old) + Δwi
where η = learning rate (small positive value, e.g., 0.1)
```

Perceptron Convergence Theorem: If the data is LINEARLY SEPARABLE, the perceptron learning algorithm is guaranteed to converge to a correct solution in finite steps.

Can a Single Perceptron Implement XOR?

Answer: NO. A single perceptron can only learn LINEARLY SEPARABLE functions — it can only draw ONE straight line to separate two classes.

XOR Truth Table:

x1	x2	XOR Output	Class
0	0	0	Class 0
0	1	1	Class 1
1	0	1	Class 1
1	1	0	Class 0

Why XOR is NOT linearly separable: Plot the 4 points: (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→0. There is NO single straight line that can separate the 0-class points from the 1-class points. XOR requires a NON-LINEAR boundary.

Neural Network for XOR (Multi-layer Solution):

Solution: Use a 2-layer network: 2 input neurons → 2 hidden neurons → 1 output neuron

```
Input Layer:  x1, x2
Hidden Layer: h1 = AND gate = (x1 AND x2)
              h2 = OR gate  = (x1 OR x2)
Output Layer: y  = NAND(h1, h2) → XOR = (x1 OR x2) AND NOT(x1 AND x2)
```

```
Alternative: XOR = (x1 OR x2) AND (NOT x1 OR NOT x2)
           = (x1 OR x2) AND NAND(x1,x2)
```

Weights for XOR network:

```
Hidden neuron h1 (NAND): w1=-2, w2=-2, bias=3, threshold=0
  (0,0): 3>0 → h1=1; (0,1): 1>0 → h1=1; (1,0): 1>0 → h1=1; (1,1): -1<0 → h1=0

Hidden neuron h2 (OR):   w1=1, w2=1, bias=-0.5, threshold=0
  (0,0): -0.5<0 → h2=0; (0,1): 0.5>0 → h2=1; (1,0): 0.5>0 → h2=1; (1,1):
1.5>0 → h2=1

Output neuron (AND):     w1=1, w2=1, bias=-1.5, threshold=0
  (h1=1,h2=0): -0.5<0 → y=0 ✓  (XOR(0,0)=0)
  (h1=1,h2=1): 0.5>0  → y=1 ✓  (XOR(0,1)=1)
  (h1=1,h2=1): 0.5>0  → y=1 ✓  (XOR(1,0)=1)
  (h1=0,h2=1): -0.5<0 → y=0 ✓  (XOR(1,1)=0)
```

Conclusion: XOR requires a minimum of 2-layer (hidden layer) neural network. Single perceptron CANNOT implement XOR.

Q4. Define Hebbian learning. Use Hebbian learning algorithm to construct a Hebbian Network which performs AND function. Is Hebbian learning supervised? [3+7] (2078 Chaitra, 2073 Bhadra, 2072 Ashwin, 2071 Magh)

Answer: Hebbian Learning

Hebbian Learning Rule: Proposed by Donald Hebb (1949): 'Neurons that fire together, wire together.' When two neurons are simultaneously active, the connection between them is strengthened.

Mathematical Rule:

```
Δwij = η × xi × yj
w(new) = w(old) + Δw
where: xi = input, yj = output, η = learning rate
```

Key Principle: If input xi and output yj are both positive (fire together) → weight increases (strengthens). No error signal is used — weight updates are driven purely by activity correlation.

Is Hebbian learning supervised? NO. Hebbian learning is UNSUPERVISED — it does not use a teacher signal or error correction. Weights are updated based on the CORRELATION between inputs and outputs only, not based on the difference between actual and desired outputs.

Constructing Hebbian Network for AND Gate

AND Truth Table (using bipolar representation: -1 instead of 0):

x1	x2	Target y (AND)
1	1	1
1	-1	-1
-1	1	-1
-1	-1	-1

Add bias: x0 = 1 always

Initial weights: w0=0, w1=0, w2=0 (all zero)

Learning rate η = 1

Hebbian Weight Updates (one pass through all examples):

Pattern	x0	x1	x2	y	Δw0=η·x0·y	Δw1=η·x1·y	Δw2=η·x2·y
x1=1,x2=1,y=1	1	1	1	1	1×1×1=1	1×1×1=1	1×1×1=1
x1=1,x2=-1,y=-1	1	1	-1	-1	1×1×(-1)=-1	1×1×(-1)=-1	1×(-1)×(-1)=1
x1=-1,x2=1,y=-1	1	-1	1	-1	1×1×(-1)=-1	1×(-1)×(-1)=1	1×1×(-1)=-1
x1=-1,x2=-1,y=-1	1	-1	-1	-1	1×1×(-1)=-1	1×(-1)×(-1)=1	1×(-1)×(-1)=1

Final weight calculations:

```
w0 = 0 + 1 + (-1) + (-1) + (-1) = -2
w1 = 0 + 1 + (-1) + 1    + 1    = 2
w2 = 0 + 1 + 1    + (-1) + 1    = 2
```

Learned weights: w0=-2, w1=2, w2=2 (bias=-2, w1=2, w2=2)

Verification:

x1	x2	NET = -2 + 2x1 + 2x2	Sign(NET)	AND(x1,x2)	Correct?
1	1	-2+2+2=2	1	1	✓
1	-1	-2+2-2=-2	-1	-1	✓
-1	1	-2-2+2=-2	-1	-1	✓
-1	-1	-2-2-2=-6	-1	-1	✓

Conclusion: The Hebbian network correctly learns AND function. Weights w0=-2, w1=2, w2=2.

SECTION 3: Multilayer Perceptron & Backpropagation Algorithm

Exam Ref: 2082, 2081, 2080, 2079, 2078, 2077, 2076, 2075, 2074, 2073, 2072, 2071

Q5. What is a neural network? Explain the backpropagation algorithm with perceptron. [Most Repeated] [2+4+4] (2075 Bhadra, 2074 Bhadra, 2072 Ashwin, 2070 Magh)

Answer: Multilayer Perceptron (MLP)

MLP: A Multilayer Perceptron is a feedforward neural network with one or more HIDDEN LAYERS between the input and output layers. Each layer is fully connected to the next. MLPs can learn non-linear decision boundaries — unlike a single perceptron.

Architecture:

- Input Layer: Receives raw features (no computation).
- Hidden Layer(s): Applies weighted sum + activation function. Learns intermediate representations.
- Output Layer: Produces final prediction.

Forward Propagation: Input flows forward through layers. At each neuron: NET = Σ(wi × xi); Output = f(NET) where f is the activation function.

Backpropagation Algorithm

Purpose: Train a multilayer neural network by propagating the ERROR BACKWARDS from output to input, adjusting weights to minimize the error using gradient descent.

Key Concept — Gradient Descent: Weights are adjusted in the direction that reduces the error (the downhill direction of the error surface).

Backpropagation Steps:

1. INITIALIZATION: Initialize all weights randomly (small values near 0).

2. FORWARD PASS: Feed input forward through the network:

```
For each layer l:
  NETj = Σ(wij × xi) + bias
  yj = sigmoid(NETj) = 1 / (1 + e^(-NETj))
```

3. COMPUTE OUTPUT ERROR:

```
Error E = ½ × Σ(target_k - output_k)²
δk (output neuron) = (target_k - output_k) × output_k × (1 - output_k)
[derivative of sigmoid: σ'(x) = σ(x)(1-σ(x))]
```

4. BACKPROPAGATE ERROR to hidden layers:

```
δj (hidden neuron) = yj × (1-yj) × Σ(δk × wjk)
[error is weighted sum of next-layer errors × local derivative]
```

5. UPDATE WEIGHTS:

```
Δwij = η × δj × xi
wij(new) = wij(old) + Δwij
where η = learning rate (e.g., 0.1 to 0.9)
```

6. REPEAT: Go to step 2 for the next training example. Continue until error is below threshold or max epochs reached.

Worked Example: Forward + Backpropagation (from 2082 Kartik / 2081 Chaitra exam)

Network: x1=-1, x2=-1. r1=max(c·x1+e·x2, 0), r2=max(d·x1+f·x2, 0). s1,s2,y use sigmoid. Weights: a=1,b=1,c=4,d=1,e=2,f=2.

```
FORWARD PROPAGATION:
r1 = max(c·x1 + e·x2, 0) = max(4×(-1) + 2×(-1), 0) = max(-6, 0) = 0
r2 = max(d·x1 + f·x2, 0) = max(1×(-1) + 2×(-1), 0) = max(-3, 0) = 0

s1 = sigmoid(a·r1) = sigmoid(1×0) = sigmoid(0) = 1/(1+e^0) = 0.5
s2 = sigmoid(b·r2) = sigmoid(1×0) = sigmoid(0) = 0.5

y  = sigmoid(s1 + s2) = sigmoid(0.5 + 0.5) = sigmoid(1) = 1/(1+e^(-1)) ≈ 0.731

OUTPUT y ≈ 0.731

WEIGHT UPDATE for second hidden layer (a and b):
Assume target = 1 (for demonstration)
Error at output: δy = y(1-y)(target-y) = 0.731×0.269×(1-0.731) =
0.731×0.269×0.269 ≈ 0.0529

Δa = η × δy × s1 = 0.1 × 0.0529 × 0.5 ≈ 0.00265
a(new) = 1 + 0.00265 ≈ 1.00265

Δb = η × δy × s2 = 0.1 × 0.0529 × 0.5 ≈ 0.00265
b(new) = 1 + 0.00265 ≈ 1.00265
```

Q6. When do we need backpropagation algorithm? Explain with an example. What is a perceptron? [2+6] (2079 Jestha, 2078 Chaitra)

Answer: When to Use Backpropagation

We need backpropagation when:

1. The problem is NON-LINEARLY SEPARABLE — simple perceptron fails, multi-layer network is needed.

2. We need to train a deep neural network with hidden layers — the gradient must flow backwards through hidden layers.

3. The relationship between input and output is complex and unknown — backprop discovers the internal representation automatically.

4. Classification or regression on complex datasets — images, speech, text.

Examples: Image recognition (millions of pixels → class label), speech recognition, natural language processing, medical image analysis, game playing.

Key insight: The chain rule of calculus allows us to compute how each weight contributes to the overall error, even through many layers — this is the mathematical foundation of backpropagation.

SECTION 4: Hopfield Network (Recurrent Neural Network)

Exam Ref: 2080 Ashwin Back, 2079 Jestha, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2075 Bhadra

Q7. When do we need a Hopfield Neural Network? Make a comparison between feedforward network and Hopfield Network. Explain with an example. [3+5] (2080 Ashwin Back, 2079 Jestha)

Answer: Hopfield Network

Hopfield Network (John Hopfield, 1982): A RECURRENT neural network where every neuron is connected to every OTHER neuron (fully connected, bidirectional, no self-connections). Unlike feedforward networks, the output feeds BACK as input — creating a dynamical system that evolves until it reaches a stable state (energy minimum).

Key Concept — Energy: Every state of the network has an associated ENERGY. The network evolves to MINIMIZE this energy — it always moves toward lower-energy states (like a ball rolling downhill).

```
Energy E = -½ Σ(i≠j) wij·si·sj - Σi θi·si
where si = state of neuron i, wij = weight between i and j, θi = threshold
```

When do we need Hopfield Networks?

- Content-Addressable Memory (CAM): Store patterns and retrieve them from partial/noisy versions.
- Pattern Completion: Given a partial pattern, complete the full stored pattern.
- Optimization Problems: Finding minimum energy configurations (e.g., TSP).
- Associative Memory: 'Given a partial query, recall the closest stored memory.'

Feedforward Network vs Hopfield Network

Feature	Feedforward Network	Hopfield Network
Architecture	Layers (input→hidden→output); one direction	Fully connected; bidirectional recurrent
Information flow	One direction: input→output	Circular: output feeds back as input
Stability	Always stable (no feedback loops)	Converges to stable states (energy minima)
Memory	No associative memory	Acts as associative/content-addressable memory
Training	Backpropagation	Hebbian rule: wij = Σ(si·sj) across stored patterns
Use case	Classification, regression, prediction	Pattern storage/retrieval, optimization
Dynamics	Static computation	Dynamic — evolves over time until stable
Example	Image classifier	Face recognition from partial/noisy image

Hopfield Network Example — Storing and Retrieving a Pattern

Pattern storage: Store two binary patterns: P1 = [1,1,-1,-1] and P2 = [1,-1,1,-1]

```
Weight matrix W: wij = Σ(p_k_i × p_k_j) for k patterns, wii = 0

W = P1^T × P1 + P2^T × P2  (with diagonal = 0)

For n=4 neurons:
W12 = (1)(1) + (1)(-1) = 0
W13 = (1)(-1) + (1)(1) = 0
W14 = (1)(-1) + (1)(-1) = -2
W23 = (1)(-1) + (-1)(1) = -2
W24 = (1)(-1) + (-1)(-1) = 0
W34 = (-1)(-1) + (1)(-1) = 0
```

Retrieval: Start with noisy pattern S = [1,1,-1,1] (last bit flipped from P1)

```
Update neuron 4: NET4 = W14·s1 + W24·s2 + W34·s3 = -2(1) + 0(1) + 0(-1) = -2
s4 = sign(-2) = -1  ← corrected to match P1!
Network converges to P1 = [1,1,-1,-1] ✓
```

SECTION 5: Kohonen Network (SOM) & Adaline Network

Exam Ref: 2079 Jestha, 2077 Chaitra, 2075 Bhadra, 2074 Bhadra, 2071 Magh, 2070 Magh

Q8. What is the role of perception in neural network? What is an Adaline network? How is it different from a perceptron? [3+5] (2070 Magh, 2071 Magh)

Answer: Adaline Network

Adaline (Adaptive Linear Neuron): Developed by Bernard Widrow and Marcian Hoff (1960). Adaline is similar to a perceptron but uses the CONTINUOUS output (NET value) — NOT the thresholded binary output — during training. This makes weight updates smoother and more stable.

Key Difference — Training Signal:

- Perceptron: error = target - sign(NET) → uses THRESHOLDED output
- Adaline: error = target - NET → uses RAW (pre-threshold) output

Adaline Learning Rule (Widrow-Hoff / Least Mean Squares):

```
error = target - NET = target - Σ(wi·xi)
Δwi = η × error × xi
wi(new) = wi(old) + Δwi
```

Advantage over Perceptron: Adaline minimizes MEAN SQUARED ERROR directly using gradient descent on the continuous output — more mathematically rigorous and often converges faster.

Perceptron vs Adaline — Comparison

Feature	Perceptron	Adaline
Proposed by	Frank Rosenblatt (1957)	Widrow & Hoff (1960)
Error computation	Uses thresholded output: error = target - sign(NET)	Uses continuous output: error = target - NET
Learning rule	Perceptron learning rule (error correction)	Widrow-Hoff / LMS rule (gradient descent on MSE)
Activation for training	Step function applied before weight update	No activation during training (uses NET directly)
Activation for prediction	Step function	Step function (same as perceptron at inference)
Convergence	Converges if data is linearly separable	Converges to minimum MSE regardless of separability
Mathematical basis	Geometric (hyperplane)	Gradient descent on error surface
Multilayer version	Multilayer Perceptron (MLP)	Madaline (Multiple Adaline)

Q9. What is the Kohonen network? Explain all the steps involved in Kohonen/SOM with suitable example. [2+6] (2079 Jestha, 2077 Chaitra, 2075 Bhadra)

Answer: Kohonen Self-Organizing Map

Kohonen Network (SOM): Developed by Teuvo Kohonen (1982). An unsupervised competitive learning network that creates a 2D map of high-dimensional data, preserving topological relationships — similar input vectors are mapped to nearby neurons on the 2D grid.

Key idea: Neurons COMPETE to respond to each input — the winner adjusts its weights to be more similar to the input, and drags its neighbors along (neighborhood learning).

SOM Algorithm Steps:

1. INITIALIZE: Set up a 2D output grid (e.g., 5×5 = 25 neurons). Initialize all weight vectors w_j randomly.

2. SELECT INPUT VECTOR x from training data (randomly).

3. FIND BMU (Best Matching Unit): The neuron j* whose weight vector is CLOSEST to x:

```
j* = argmin_j ||x - w_j||  (Euclidean distance)
```

4. UPDATE WEIGHTS: Move BMU and its neighbors closer to x:

```
w_j(t+1) = w_j(t) + η(t) × h(j*,j,t) × [x - w_j(t)]
where: η(t) = learning rate (decreases with time t)
       h(j*,j,t) = neighborhood function (Gaussian, decreases with distance
and time)
h(j*,j,t) = exp(-||r_j* - r_j||² / (2σ(t)²))
σ(t) = σ□ × exp(-t/τ)  ← neighborhood radius shrinks over time
```

5. REDUCE η(t) and σ(t) (cooling schedule).

6. REPEAT from Step 2 until max iterations or convergence.

Applications:

- Visualizing high-dimensional data in 2D (like a map).
- Document clustering — similar documents appear in nearby map locations.
- Customer segmentation, anomaly detection.
- Dimensionality reduction.

SECTION 6: Expert Systems — Architecture, Development & Knowledge Acquisition

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Chaitra, 2080 Ashwin Back, 2079 Chaitra, 2079 Jestha, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Magh

Q10. Define term expert system and expert system Shells. What are the characteristics that problems possess to be solved by expert system? Explain with example. **[2+2+3]**
(2082 Kartik, 2081 Chaitra, 2079 Chaitra)

Answer: Expert System

Expert System: An Expert System (ES) is an AI program that emulates the decision-making ability of a human EXPERT in a specific domain. It contains a large body of domain-specific knowledge and applies reasoning (inference) to provide expert-level recommendations, diagnoses, or solutions.

Classic Examples:

- MYCIN (1970s): Diagnoses bacterial infections and recommends antibiotics — Stanford University.
- DENDRAL (1965): Identifies chemical compound structures — Stanford.
- XCON (1980s): Configures DEC VAX computer systems.
- PROSPECTOR: Identifies mineral deposits for geological exploration.

Expert System Shell: An Expert System Shell is an EMPTY Expert System framework — it provides the architecture (inference engine, knowledge base interface, explanation facility) but WITHOUT any domain knowledge. Knowledge engineers fill it with domain-specific rules to create a complete ES. Example shells: CLIPS, JESS, Prolog.

Characteristics of Problems Solvable by Expert Systems:

Characteristic	Description	Example
Requires genuine expertise	Problem requires years of human expertise to solve	Medical diagnosis, legal advice, tax planning
Domain is well-defined	Knowledge is limited to a specific domain, not general commonsense	Only diagnoses bacterial infections, not all diseases
Knowledge is available	Experts exist and can articulate their knowledge as rules	Physicians can write IF-THEN diagnosis rules
Primarily symbolic	Problem involves reasoning with symbols/rules, not just math	Rule-based diagnosis, not numerical simulation
High stakes	Worth the effort to encode expert knowledge	Medical, financial, legal domains
Requires explanation	Users need to understand why a decision was made	Patients want to know why they were diagnosed
Changing knowledge base	Expert knowledge needs to be updated over time	New drug interactions, new tax laws

Q11. Define Expert System with example. Explain the limitations of Expert System. [4+3] (2081 Chaitra, 2077 Chaitra)

Answer: Limitations of Expert Systems

Limitation	Explanation	Impact
Knowledge Acquisition Bottleneck	Extracting and encoding expert knowledge is slow, expensive, and difficult	High development cost and time
Narrow Domain	ES works ONLY within its specific domain — cannot generalize to related areas	MYCIN cannot diagnose cancer even if symptoms overlap
Cannot Learn	Traditional ES cannot learn from new cases — knowledge must be manually updated	Becomes outdated as field evolves
No Common Sense	Lacks basic commonsense reasoning that humans take for granted	May give absurd recommendations in edge cases
Poor Explanation of Deep Reasoning	Can explain WHAT rules fired, but not WHY those rules make sense	Users may not trust the explanation
Handling Uncertainty	Rule-based ES struggles with probabilistic, vague, or incomplete information	Hard to represent 'probably' or 'maybe' in rigid IF-THEN rules
High Maintenance	As domain knowledge changes, rules must be manually revised	Costly ongoing maintenance
Brittleness	Performance degrades sharply near the boundary of its knowledge	One wrong assumption can cascade to wrong conclusion

Q12. Draw the block diagram of an expert system and briefly explain each component. What are the benefits? [6+2] (2077 Chaitra, 2074 Bhadra)

Answer: Architecture of an Expert System

Block Diagram Components:

```
┌──────────────────────────────────────────────────────────────┐
│                    EXPERT SYSTEM                             │
│                                                              │
│  [USER]                                                      │
│     │                                                        │
│     ▼                                                        │
│  [USER INTERFACE] ←──────────────────── [EXPLANATION         │
│     │                                     FACILITY]          │
│     ▼                                         ▲              │
│  [INFERENCE ENGINE] ──────────────────────────│              │
│     │           │                             │              │
│     ▼           ▼                             │              │
```

```
│  [KNOWLEDGE  [WORKING                    [KNOWLEDGE          │
│   BASE]       MEMORY]                     ACQUISITION        │
│     ▲                                     SUBSYSTEM]         │
│     │                                         │              │
│  [KNOWLEDGE ENGINEER] ──────────────────────  │              │
│                                [DOMAIN EXPERT] ─┘             │
└──────────────────────────────────────────────────────────────┘
```

Component	Description	Example (Medical ES)
Knowledge Base	Stores domain-specific IF-THEN rules and facts	IF fever>38 AND cough THEN suspect flu
Working Memory	Stores current facts about the specific case	Patient: fever=39, cough=yes, age=30
Inference Engine	Applies rules to working memory facts to derive conclusions; forward or backward chaining	Matches patient symptoms to rules → diagnosis
Knowledge Acquisition	Subsystem for adding/modifying/validating knowledge	Physician inputs new drug interaction rules
Explanation Facility	Explains HOW and WHY a conclusion was reached	'Flu was diagnosed because fever>38 AND cough'
User Interface	Natural language or form-based interface for user interaction	Doctor enters symptoms via GUI

Benefits: Available 24/7, consistent decisions, can train new experts, captures retiring experts' knowledge, fast response, handles complex rule sets.

Q13. You need to work as a Knowledge Engineer for a company developing an expert system for Disease Diagnosis. Draw a block diagram and highlight the major steps for development of this expert system. [6] (2080 Chaitra)

Answer: Development Stages of an Expert System

Stage	Description	Activities for Disease Diagnosis ES
1. Problem Identification	Define the problem scope, goals, and feasibility	Decide: diagnose bacterial infections in urban clinics
2. Knowledge Acquisition	Gather domain knowledge from experts, books, databases	Interview physicians; review medical literature; encode 500+ diagnosis rules
3. Knowledge Representation	Encode knowledge in formal structure (rules, frames)	IF symptom THEN disease (production rules); frame for each disease
4. Prototype Development	Build small working prototype with limited knowledge	ES for 10 common diseases; test with 50 test cases
5. Testing & Validation	Test against known cases; compare with expert judgment	Compare ES diagnosis with specialist diagnosis on 500 cases; accuracy ≥85%

6. Knowledge Refinement	Identify gaps and errors; refine rules iteratively	Add rare disease rules; fix misdiagnosis cases
7. Deployment	Deploy to users; integrate with hospital systems	Hospital doctors use ES via tablet interface
8. Maintenance	Update knowledge as medicine evolves; add new diseases	Add COVID-19 diagnosis rules; update drug interaction database

SECTION 7: Natural Language Processing (NLP) — Levels, Steps & Challenges

Exam Ref: 2082 Kartik, 2081 Ashwin, 2081 Chaitra, 2080 Chaitra, 2079 Chaitra, 2079 Jestha, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2075 Baisakh, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin

Q14. What are the problems associated with NLP? List down the different steps involved in natural language processing with suitable example. [Most Repeated] **[2+5]** (2082 Kartik, 2081 Chaitra, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra)

Answer: Natural Language Processing (NLP)

NLP: Natural Language Processing is the branch of AI that enables computers to UNDERSTAND, INTERPRET, and GENERATE human language (text and speech) in a meaningful and useful way.

Goals:

- NLU (Natural Language Understanding): Computer understands what the human said.
- NLG (Natural Language Generation): Computer produces human-readable text.

Problems / Challenges Associated with NLP:

Problem	Description	Example
Ambiguity (Lexical)	A word has multiple meanings	'Bank' → river bank OR financial bank
Ambiguity (Syntactic)	A sentence can be parsed in multiple ways	'I saw the man with the telescope' — who has the telescope?
Ambiguity (Semantic)	Sentence meaning is unclear	'Flying planes can be dangerous' — planes flying OR the act of flying
Anaphora Resolution	Identifying what a pronoun refers to	'John told Bill he was smart.' — who is smart?
Idioms & Figurative Language	Literal meaning ≠ intended meaning	'It's raining cats and dogs' doesn't involve animals
Sarcasm & Irony	Text means the opposite of what it says	'Oh great, another Monday!' — not actually great
Context Dependency	Meaning depends on context	'Can you pass the salt?' is a request, not a yes/no question
Multilingual Challenges	Languages have different grammar, script, structure	Nepali has postpositions; English has prepositions
Spelling & Grammar Errors	Real text is noisy	'teh' instead of 'the'; 'their/there/they're' confusion

Steps / Levels of NLP Analysis:

1. PHONOLOGICAL ANALYSIS (Phonetics/Phonemics)

Concerned with sounds and pronunciation of words.

- Converts spoken language into phonemes (sound units).
- Example: 'Cat' /k/ /æ/ /t/ → 3 phonemes
- Used in: Speech recognition, text-to-speech systems.

2. MORPHOLOGICAL ANALYSIS

Studies the structure of words — how words are formed from smaller units (morphemes).

- Breaks words into root forms + affixes.
- Processes: Stemming (running→run), Lemmatization (better→good).
- Example: 'Unhappiness' → un- (prefix) + happy (root) + -ness (suffix)

3. LEXICAL ANALYSIS (Tokenization)

Breaks text into individual TOKENS (words, punctuation) and identifies their properties.

- Part-of-Speech (POS) Tagging: Labels each word as noun, verb, adjective, etc.
- Example: 'The cat sat.' → [The/DET] [cat/NOUN] [sat/VERB] [./PUNCT]

```
Input: 'She quickly solved the problem'
POS:   PRN  ADVERB VERB   DET  NOUN
```

4. SYNTACTIC ANALYSIS (Parsing)

Analyzes the GRAMMATICAL STRUCTURE of a sentence — checks grammar rules and creates a parse tree.

- Context-Free Grammar (CFG) is commonly used.
- Parse tree shows how words combine into phrases and sentences.

```
'The cat sat on the mat'
S → NP VP
NP → Det N → 'The cat'
VP → V PP → 'sat on the mat'
PP → P NP → 'on the mat'
```

5. SEMANTIC ANALYSIS

Determines the LITERAL MEANING of the sentence — mapping syntactic structures to meanings.

- Word Sense Disambiguation: Which meaning of an ambiguous word applies?
- Semantic Role Labeling: Who did what to whom (Agent, Patient, Location).
- Example: 'Bank charges interest' → bank = financial institution (not river bank)

6. DISCOURSE ANALYSIS (Pragmatic / Context)

Analyzes MEANING ACROSS SENTENCES — understanding references, context, discourse coherence.

- Coreference Resolution: Linking pronouns to their referents.
- Example: 'John is a programmer. He loves Python.' → 'He' refers to John.

7. PRAGMATIC ANALYSIS

Understands the INTENDED MEANING — what the speaker means, not just the literal words.

- Speech Acts: Request, assertion, question, promise, etc.
- Example: 'Can you pass the salt?' → This is a REQUEST, not a question about ability.
- Cultural and situational context matters here.

Q15. You are hired by a company to work on a Nepali Language product. What are the challenges/issues associated with NLP? List them with examples and relate to Nepali Language. **[8]** (2081 Ashwin)

Answer: NLP Challenges Specific to Nepali Language

Challenge	General Problem	Nepali-Specific Challenge
Script Complexity	Non-ASCII scripts are harder to process	Devanagari script; multiple Unicode representations of same character; half-letters (हलन्त)
Limited Resources	Less training data than English	Very few large labeled Nepali corpora; limited pretrained models
Morphological Richness	Complex word forms	Nepali verbs change for gender, number, tense, politeness (ग्यो/गइन्/गए)
Code-Mixing	Mixing of languages in text	Nepali text often mixes English words: 'मैले computer use गरें'
Dialects	Multiple dialects with different vocabulary	Bajhang, Palpa, Kathmandu dialects vary significantly
Postpositions	Different structure from English	'घरमा' (at home) vs English 'at home' — postpositions in Nepali
Ambiguity	Homographs with different meanings	'काल' = time OR death; context determines meaning
Lack of Tools	Few NLP libraries support Nepali	No mature Nepali equivalent of NLTK/SpaCy

Mitigation strategies: Transfer learning from multilingual models (mBERT, XLM-R), creating annotated Nepali corpora, rule-based preprocessing for Devanagari script normalization, developing Nepali WordNet.

SECTION 8: Machine Vision (Introduction to Computer Vision)

Exam Ref: 2081 Ashwin, 2080 Ashwin Back, 2080 Chaitra, 2079 Chaitra, 2079 Jestha, 2078 Chaitra, 2074 Bhadra

Q16. What is Machine Vision? List down the steps involved in Machine Vision by giving a suitable example. [2+4] (2081 Ashwin, 2080 Chaitra, 2079 Chaitra)

Answer: Machine Vision

Machine Vision (Computer Vision): Machine Vision is the field of AI that enables computers to INTERPRET and UNDERSTAND visual information from the real world (images, videos). It gives machines the ability to 'see' — just as human vision allows us to perceive, recognize, and reason about the visual world.

Applications:

- Quality control in manufacturing (detect defects in products)
- Medical imaging (X-ray analysis, tumor detection, pathology slides)
- Autonomous vehicles (lane detection, obstacle recognition)
- Face recognition and surveillance
- Agricultural monitoring (crop health from drones)
- Augmented Reality and Robotics

Steps in Machine Vision:

Step	Description	Example (Defect Detection in Factory)
1. Image Acquisition	Capture digital image using camera, scanner, or sensor	Industrial camera captures image of manufactured circuit board
2. Pre-processing	Enhance image quality: noise removal, contrast adjustment, resizing	Apply Gaussian blur to reduce sensor noise; normalize brightness
3. Segmentation	Divide image into meaningful regions or objects	Separate circuit board from background; identify individual components
4. Feature Extraction	Extract relevant features: edges, corners, textures, shapes, color	Extract shape of each solder joint; measure circularity
5. Object Recognition	Match extracted features to known objects or classes	Compare solder joint shape to template of 'good' vs 'defective' joints
6. Interpretation	Apply domain knowledge to make a decision	IF shape deviation > 0.1mm THEN flag as defective
7. Output / Action	Produce result and trigger appropriate action	Mark defective board; activate robotic arm to remove it from line

Another Example: Autonomous Vehicle

Step	Process in Self-Driving Car
Image Acquisition	Cameras, LIDAR, radar capture 360° view of surroundings
Pre-processing	De-noise images; correct for lens distortion; normalize lighting

Segmentation	Separate road, lane markings, vehicles, pedestrians, signs
Feature Extraction	Extract lane line geometry, vehicle bounding boxes, traffic sign shapes
Recognition	Identify: pedestrian crossing, red traffic light, school zone sign
Interpretation	'Red light ahead + pedestrian in crosswalk' → must stop
Output/Action	Apply brakes; maintain safe following distance; adjust steering

QUICK REVISION: Chapter 7 — Key Points Summary (26 Marks!)

Topic	Key Points to Remember	Expected Marks
McCulloch-Pitts Neuron	Binary inputs, fixed weights, threshold θ; no learning; AND gate example	3-4
Activation Functions	Step, Sigmoid, Tanh, ReLU, Softmax; how to choose for each layer/task	3-4
Perceptron	Trainable; $\Delta w=\eta(t-y)x$; convergence if linearly separable; XOR needs 2 layers	4-6
XOR with MLP	2 inputs → 2 hidden (NAND+OR) → 1 output (AND); show weights that work	6-8
Hebbian Learning	Unsupervised; $\Delta w=\eta\cdot x\cdot y$; fire together→wire together; AND gate example	4-6
Backpropagation	Forward pass → compute error → δ output → δ hidden → update weights	6-8
Hopfield Network	Recurrent, fully connected; energy minimization; associative memory; CAM	4-6
Adaline	Uses NET (not output) for training; LMS rule; smoother than perceptron	3-4
SOM/Kohonen	Unsupervised; BMU; neighborhood update; topology preservation; 2D map	3-4
Expert System	Knowledge Base + Inference Engine + Explanation; MYCIN example; 4 development stages	6-8
ES Limitations	No learning, narrow domain, knowledge bottleneck, no commonsense	3-4
NLP Steps	Phonological→Morphological→Lexical→Syntactic→Semantic→Discourse→Pragmatic	5-7
NLP Problems	Ambiguity (lexical/syntactic/semantic), anaphora, sarcasm, context-dependency	3-4
Machine Vision Steps	Acquire→Preprocess→Segment→Extract Features→Recognize→Interpret→Act	4-6

Best of Luck! | TU IOE BCT | CT 653 AI | Chapter 7: Applications of AI (Neural Networks + Expert Systems + NLP + Machine Vision)